RAPPORT DE TÂCHE

Mission 5 — Tâche 3 : Intégrer des logs

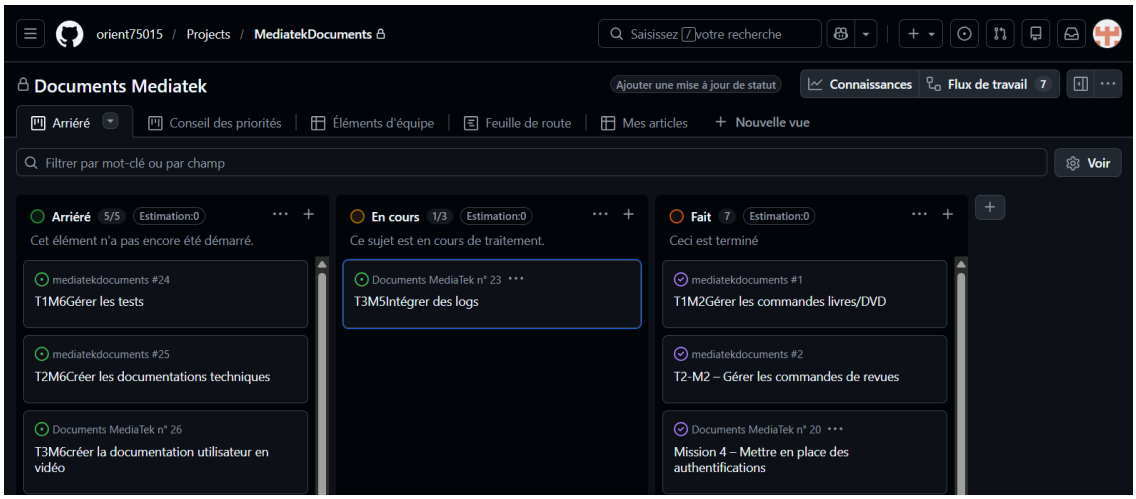
Projet MediatekDocuments — C# / NLog

Projet	MediatekDocuments — BTS SIO SLAM
Mission	Mission 5 — Sécurité et qualité
Tâche	Tâche 3 — Intégrer des logs
Temps estimé	1 heure
Temps réel	1 heure
Bibliothèque utilisée	NLog 6.1.2 (installé via NuGet)
Fichiers modifiés/créés	dal\Access.cs, NLog.config (nouveau)

1. Demande de la tâche

Dans la classe Access, ajouter le code de configuration des logs et des logs au niveau de chaque affichage console, afin d'enregistrer les erreurs et avertissements dans un fichier de logs.

2. Kanban



Capture du Kanban GitHub — T3M5 dans la colonne 'In Progress'

3. Temps estimé et temps réel

Indicateur	Valeur
Temps estimé	1 heure
Temps réel	1 heure
Écart	Aucun

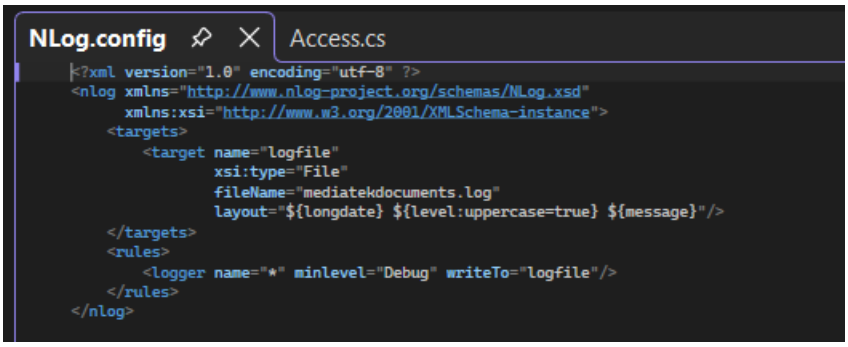
4. Configuration des logs — NLog.config

La bibliothèque NLog a été installée via NuGet (version 6.1.2). La configuration des logs est définie dans le fichier NLog.config placé à la racine du projet, avec la propriété 'Copier dans le répertoire de sortie' définie sur 'Toujours copier'.

Contenu du fichier NLog.config :

```
<?xml version="1.0" encoding="utf-8" ?>
<nlog xmlns="http://www.nlog-project.org/schemas/NLog.xsd"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <targets>
    <target name="logfile"
            xsi:type="File"
            fileName="mediatekdocuments.log"
            layout="${longdate} ${level:uppercase=true} ${message}"/>
  </targets>
  <rules>
    <logger name="*" minlevel="Debug" writeTo="logfile"/>
  </rules>
</nlog>
```

Explication : les logs sont écrits dans le fichier mediatekdocuments.log avec le format : date/heure, niveau (DEBUG/INFO/WARN/ERROR), message. Tous les niveaux à partir de Debug sont enregistrés.



Capture du fichier NLog.config dans Visual Studio

5. Déclaration du logger dans Access.cs

Dans la classe Access, le logger est déclaré comme champ statique en utilisant la méthode GetCurrentClassLogger() de NLog :

```
using NLog;

private static readonly Logger logger = LogManager.GetCurrentClassLogger();
```

```
1   using System;
2   using System.Collections.Generic;
3   using MediaTekDocuments.model;
4   using MediaTekDocuments.manager;
5   using Newtonsoft.Json;
6   using Newtonsoft.Json.Converters;
7   using Newtonsoft.Json.Linq;
8   using System.Configuration;
9   using System.Linq;
10  using NLog;
11
12  namespace MediaTekDocuments.dal
13  {
        6 références | orient75015, Il y a 16 minutes | 1 auteur, 7 modifications
14      public class Access
15      {
16          private static readonly string uriApi = System.Configuration.ConfigurationManager.AppSettings["uriApi"];
17          private static Access instance = null;
18          private readonly ApiRest api = null;
19          private const string GET = "GET";
20          private const string POST = "POST";
21          private const string PUT = "PUT";
22          private const string DELETE = "DELETE";
23          private const string CHAMPS = "champs=";
24          private static readonly Logger logger = LogManager.GetCurrentClassLogger();
```

Capture de Access.cs — using NLog et déclaration du logger (dal\Access.cs, lignes 1-24)

6. Exemples de logs ajoutés

6.1 Log de niveau Error — erreur de connexion à l'API

Dans le constructeur Access(), le Console.WriteLine a été remplacé par un logger.Error qui enregistre l'exception complète :

```
catch (Exception e)
{
    logger.Error(e, e.Message);
    Environment.Exit(0);
}
```

```
33          }
34          catch (Exception e)
35          {
36              logger.Error(e, e.Message);
37              Environment.Exit(0);
38          }
```

6.2 Log de niveau Warn — code erreur retourné par l'API

Dans la méthode TraitementRecup, le Console.WriteLine pour les codes erreur a été remplacé par un logger.Warn :

```
else
{
    logger.Warn("code erreur = " + code + " message = " + (String)retour["message"]);
}
```

```
181         else
182         {
183             logger.Warn("code erreur = " + code + " message = " + (String)retour["message"]);
184         }
185     }
```

6.3 Log de niveau Error — exception lors de l'accès à l'API

Dans la méthode TraitementRecup, l'exception générale est enregistrée avec son message :

```
catch (Exception e)
{
    logger.Error(e, "Erreur lors de l'accès à l'API : " + e.Message);
    Environment.Exit(0);
}
```

```
186         catch (Exception e)
187         {
188             logger.Error(e, "Erreur lors de l'accès à l'API : " + e.Message);
189             Environment.Exit(0);
190         }
191         return liste;
```

6.4 Log dans les méthodes CRUD

Dans toutes les méthodes d'insertion, modification et suppression (CreerExemplaire, CreerCommandeDocument, etc.), les exceptions sont loguées :

```
catch (Exception ex)
{
    logger.Error(ex, ex.Message);
}
```

```
129         catch (Exception ex)
130         {
131             logger.Error(ex, ex.Message);
132         }
133         return false;
```

7. Résultat

Les logs sont intégrés dans la classe Access. Lors de l'exécution de l'application, toutes les erreurs et avertissements sont enregistrés dans le fichier mediatekdocuments.log situé dans le dossier bin\Debug\ du projet.

- Erreurs de connexion à l'API → logger.Error
- Codes erreur retournés par l'API → logger.Warn
- Exceptions dans les méthodes CRUD → logger.Error
- Format du log : date/heure + niveau + message